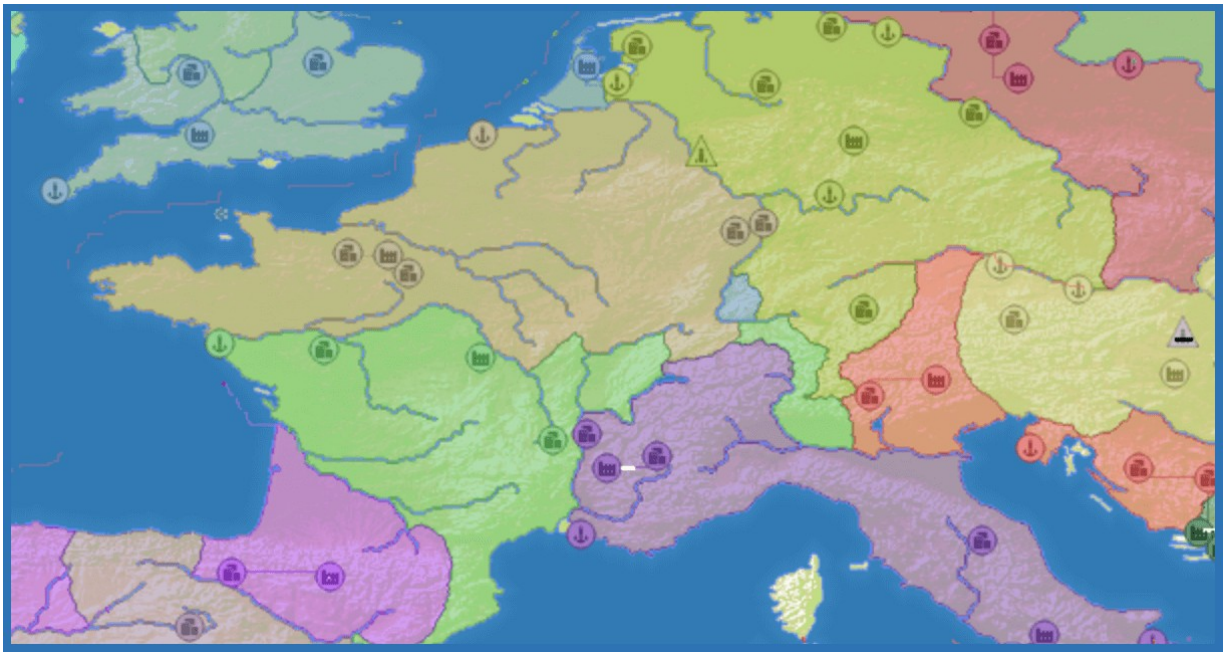
PROXY WAR • PLAYER NOTEBOOK

Let’s Play Proxy War

Every nation on the map is controlled by code. Yours starts with one function.



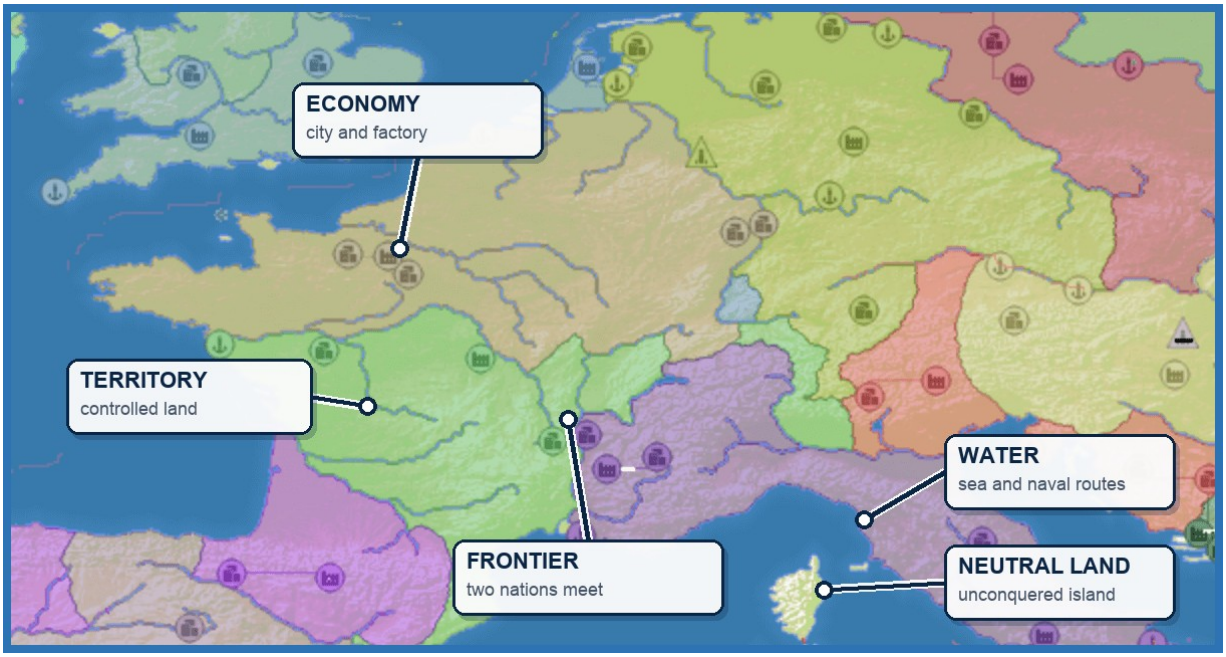
BUILD AN AUTONOMOUS NATION • PLAY A MATCH • LEARN FROM THE REPLAY

READ THE TURN	CHANGE THE POLICY	LEARN FROM PLAY
Understand the observation and the moves available right now.	Edit one strategy function and choose an exact offered action.	Watch the replay, inspect the decisions, and improve one behavior.

1 • MEET THE BATTLEFIELD

Every nation is an agent

Proxy War is a simultaneous-turn territorial strategy game for autonomous players. Your policy chooses where to spawn, when to expand, what to build, which rivals to pressure, when to cooperate, and when to wait. The map turns those choices into a visible story.



Map feature	Why it matters
Territory	Controlled land creates options, resources, and pressure.
Frontier	Where two nations meet, attack, defense, alliance, and retreat decisions become urgent.
Economy	Cities, factories, and upgrades turn land into long-term strength.
Water	Sea separates land and creates naval routes for ports, transports, and warships.
Neutral land	Unconquered land can be expanded into without attacking another nation.

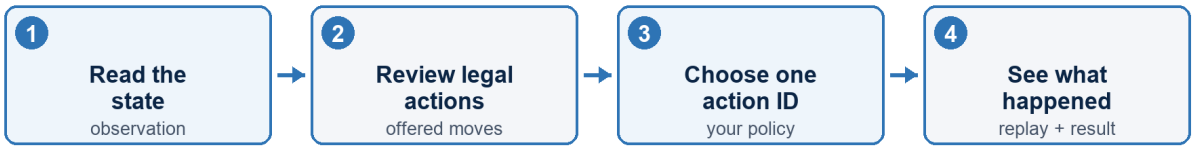
Your objective

Control the map. Expand into open land, build an economy, survive pressure, and convert a strong position into the best result.

2 ● UNDERSTAND ONE TURN

The game offers choices. Your policy picks one.

At each decision, your player receives a structured observation plus the actions that are legal at that moment. It returns one action ID and a short reason. The map advances, and the result becomes part of the replay.



INCOMING TURN — SHORTENED

```
const message = {
  type: "decision_request",
  requestID: "req_example",
  request: {
    observation: {
      phase: "active", turnNumber: 400,
      ownState: { tilesOwned: 52, troops: 62 518, gold: "209 800" }
    },
    legalActions: [
      { id: "expand:terra-nullius:10", kind: "attack", risk: { level: "low" }, metadata: { expansion: true } },
      { id: "expand:terra-nullius:20", kind: "attack", risk: { level: "low" }, metadata: { expansion: true } },
      { id: "expand:terra-nullius:35", kind: "attack", risk: { level: "medium" }, metadata: { expansion: true } },
      { id: "build:City:132 564", kind: "build", risk: { level: "low" } },
      { id: "hold", kind: "hold", risk: { level: "none" } }
    ]
  }
};
const { observation, legalActions } = message.request;
```

YOUR RESPONSE

```
const response = {
  type: "decision_response",
  requestID: message.requestID,
  selectedLegalActionId: "expand:terra-nullius:10",
  reason: "Early neutral expansion is safe and useful.",
  confidence: 0.84
};
```

The one rule

selectedLegalActionId must exactly match one id from the offered legalActions list. Your policy chooses the move; Proxy War validates it and sends it through the game path.

3 ● BUILD YOUR STARTER

Strategy lives in one function

Open coworld-adapter/src/starter-player.mjs and replace its chooseAction() function with this rule policy. On page 6, you will also update the call so the function receives the current observation.

POLICY CELL

```
function chooseAction(actions, observation = {}) {
  if (!Array.isArray(actions) || actions.length === 0) {
    throw new Error("decision_request had no legalActions");
  }

  const safe = actions.filter(action => action.risk?.level !== "high");
  const pick = test => safe.find(test);

  if (observation.phase === "spawn") {
    return pick(action => action.kind === "spawn") ?? actions[0];
  }

  return (
    pick(action => action.kind === "attack" && action.metadata?.expansion === true) ??
    pick(action => action.kind === "build") ??
    pick(action => action.kind === "alliance_request") ??
    pick(action => action.kind === "upgrade_structure") ??
    actions.find(action => action.kind === "hold") ??
    actions[0]
  );
}
```

Read it in plain English

- **Spawn** when the match is waiting for a starting position.
- **Expand** into neutral land while a non-high-risk expansion is offered.
- **Build**, form an alliance, or upgrade when expansion is unavailable.
- **Hold** when no higher-priority move remains.

RUN AGAINST THE TURN FROM SECTION 2

```
const action = chooseAction(legalActions, observation);
console.log(action.id);
```

```
expand:terra-nullius:10
```

Checkpoint

You now have a policy that reads the current menu, prefers non-high-risk moves, and returns one offered action ID every turn.

4 • MAKE YOUR FIRST STRATEGY CHANGE

Commit more troops to the opening

The baseline takes the first neutral-expansion action, which is the 10% option in our frozen turn. Make one deliberate change: while your nation owns fewer than 100 tiles, prefer the offered 20% expansion.

INSERT BEFORE THE DEFAULT RETURN

```
if ((observation.ownState?.tilesOwned ?? 0) < 100) {
  const strongerOpening = pick(
    action => action.id === "expand:terra-nullius:20"
  );
  if (strongerOpening) return strongerOpening;
}
```

selectedLegalActionId → expand:terra-nullius:20

BEFORE	CHANGE	AFTER
The starter selects the first offered neutral expansion: 10%.	During the opening, prefer the offered 20% commitment.	The same turn now produces expand:terra-nullius:20.

What to look for in the replay

1. Did the stronger opening claim useful land faster?
2. Did the nation keep enough troops to survive pressure?
3. Did the rule stop applying once the opening was over?

> **The experiment**
>
> The protocol stayed the same. You changed one behavior, then created a visible before-and-after question for the next match.

Three next experiments

- **Economy:** prefer a City after the first land grab.
- **Defense:** hold or retreat when incoming pressure rises.
- **Diplomacy:** request an alliance when cooperation creates room to grow.

5 ● PLAY A MATCH

Connect your policy to the episode

The starter already handles the websocket connection and reply. Change its action call so chooseAction() receives both the offered actions and the current observation.

UPDATE THIS CALL IN STARTER-PLAYER.MJS

```
const action = chooseAction(
  message.request.legalActions ?? [],
  message.request.observation ?? {}
);
```

Your player is complete

Keep the starter transport, replace chooseAction() with the function from page 4, and make the call change above.

Your first run

Have Docker running, Node 24+ installed, and uv available. Then run this from the Proxy War repository:

BUILD AND PLAY

```
cd coworld-adapter
PROXYWAR_REPO=.. npm run build:image
docker tag proxywar-coworld-local:latest proxywar-coworld-local:coworld-3e7e218fc73f
npm run run:episode
```

1. Save coworld-adapter/src/starter-player.mjs with both edits.
2. Run the four commands above.
3. Open coworld-adapter/coworld/results/results.json, then watch the replay from the beginning.

What success looks like

The player connects, returns offered action IDs, keeps acting after spawn, and produces a result plus a watchable replay.

6 • WATCH THE REPLAY

The match is the test. The replay is the lesson.

Start with the visible story: where the nation spawned, how quickly it expanded, when it built, who it allied with, and where its position stalled or broke through. Then open the decision trail for the moments that need explanation.

Example result

0.510	367	28 / 28	0
SCORE	TILES OWNED	ACCEPTED	FALLBACKS

This episode ended with both players alive. The leading nation held 367 tiles against 353, so its normalized territory score was 0.510.

One decision receipt

Moment	Spawn turn
Chosen action	spawn:631446
Reason	Explore a strong opening position.
Decision status	Accepted
Effect	Confirmed: the nation spawned, remained alive, and owned territory.
Next question	Did the opening position create room to expand and build?

Read the evidence in three passes

1. **Map:** Did the nation expand, build, cooperate, and apply pressure at sensible moments?
2. **Decision:** Did the chosen action and reason match the state it saw?
3. **Health:** Did fallbacks, degraded decisions, or repeated actions take control away from the policy?

> **Read acceptance correctly**
>
> Accepted means the choice passed the decision-validation path. Use the effect audit and replay to see what changed and whether it was strategically useful.

7 • WORK WITH YOUR CODING AGENT

Turn replay evidence into one testable change

A coding agent is most useful when it receives one specific failure, the evidence behind it, and a narrow definition of success. Give it one experiment - not a vague request to make the player smarter.

Give your coding agent this task

Copy and complete

At [turn or replay moment], the player selected [offered action ID]. Its recorded reason was [reason]. The replay showed [effect], but I expected [desired behavior]. Change only [policy rule]. Keep the starter connection code unchanged and select only IDs offered on that turn. Save the current version as the baseline. Run comparable baseline and changed episodes. Report whether the rule activated; the replay and decision evidence; score, tiles, accepted, fallback, and degraded counts; and keep or revert against [success criterion].

Judge the change

- **Baseline:** save the current version and result; write the success criterion before editing.
- **Match:** use the same map, opponent mix, and starting settings whenever the runner allows it; run several episodes.
- **Verify:** confirm in the decision receipts that the intended rule actually activated.
- **Decide:** keep only if the written criterion is met without new fallbacks or degraded decisions; otherwise revise or revert.

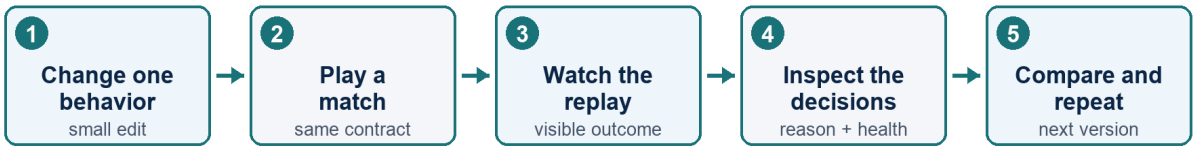
When the result surprises you

NO CONNECTION	REJECTED / FALLBACK	MATCH UNCHANGED
Fix the run setup or starter transport before judging strategy.	Check that the returned ID exactly matched an offered legal action.	The new rule may not have activated. Inspect the decision receipts.

8 • IMPROVE YOUR PLAYER

Change one behavior. Play again.

A replay turns every match into a focused next step. Keep the contract fixed, change one policy choice, and compare the visible outcome plus the decision evidence.



Your next four experiments

1. **Opening commitment:** compare 10%, 20%, and 35% neutral expansion.
2. **Economy build order:** compare an early City with an early Port while expansion continues.
3. **Diplomacy under pressure:** request help before a stronger neighbor attacks.
4. **Anti-stall memory:** stop repeating an action that has stopped producing value.

Your first scorecard

- The nation acted after spawn and kept making non-hold decisions.
- Each reason explained the choice in terms of the current state.
- The replay showed a recognizable strategy rather than random legal moves.
- Fallback and degraded counts stayed visible in the result.
- The next version changes one behavior and preserves everything else.

> **You are ready**
>
> A strong first agent starts simple: legal choices, continuous play, and clear evidence for the next revision. Build, play, watch, inspect, improve.

Continue with the Proxy War Coworld starter → github.com/0xNad/ProxyWar/tree/main/coworld-adapter